AEGIS-Q: A UMA-Aware Secure Edge File-Encryption Runtime

Changjong Kim
changjong5238@seoultech.ac.kr
Seoul National University of Science and Technology
Seoul, Republic of Korea

Ehan Sohn
ehanjsohn@seoultech.ac.kr
Seoul National University of Science and Technology
Seoul, Republic of Korea

Yongseok Son
sysganda@cau.ac.kr
Chung-Ang University
Seoul, Republic of Korea

Sunggon Kim
sunggonkim@seoultech.ac.kr
Seoul National University of Science and Technology
Seoul, Republic of Korea

Abstract

Unified-memory edge devices co-locate local databases, encrypted logs, cache-manifest updates, and encrypted state on shared DRAM/storage. A secure file-encryption runtime must place cryptographic work, not merely accelerate it: GPU work can steal bandwidth from foreground storage and durable publication. Its thesis is that secure edge storage should expose one durable authenticated format with a CPU-first AES-GCM data plane and an elastic GPU/PQC maintenance lane. AEGIS-Q explores that boundary as a boundary-aware UMA FUSE runtime with optional batched PQC key-plane maintenance, executor-local managed-memory use, and fail-closed external-anchor replay-after-advance checks. The core result is asymmetric placement under edge pressure. AES-GCM file records remain CPU-first; ML-KEM/cuPQC work uses the GPU only when it is independent, batched, and slack-tolerant; storage-visible QoS throttles elastic writes rather than weakening publication order. On a Jetson AGX Thor Developer Kit, unthrottled secure-storage pressure reports 9.62 ms SQLite p99 and 6.98 MB/s background writes; AEGIS-Q reports 8.15 ms p99, zero median misses, and 3.02 MB/s background writes, while app-only is 7.25 ms. The same mounted path exercises remount/tamper checks, generation faults, rekey fallback, selected app-crash timing, and telemetry-to-FUSE throttling. This is a scoped edge file-encryption runtime result, not deployed-filesystem or peak-throughput superiority.

Keywords

Post-Quantum Cryptography, Edge Computing, Secure Storage, Unified Memory, TPM, FUSE

1 Introduction

Edge devices increasingly run local databases, encrypted logs, cache-manifest updates, and encrypted storage on shared DRAM/storage. Confidentiality and integrity remain mandatory, but a daemon that treats the integrated GPU as a free accelerator can contend with the foreground storage operations it protects. The systems problem is therefore not filesystem conformance, application scheduling, or throughput encryption in isolation; it is a secure file-encryption runtime that makes CPU/GPU placement, storage-visible QoS, authenticated publication, and fail-closed replay checks one mounted decision surface. Its thesis is that secure edge storage should expose one durable authenticated format while separating a CPU-first AES-GCM data plane from an elastic GPU/PQC maintenance lane.

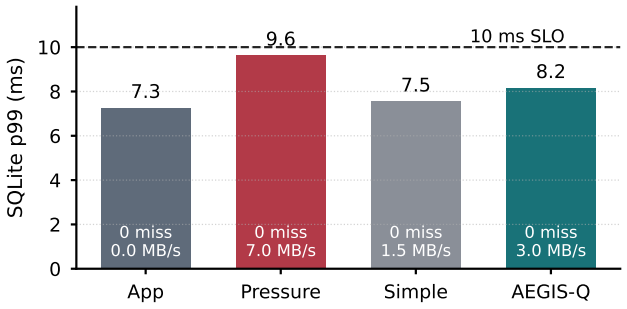

Figure 1: SQLite pressure result for RQ2; labels show deadline misses and background throughput.

Table 1: Capability comparison that defines AEGIS-Q's design gap. "Policy" means storage-visible accelerator/QoS/replay policy, not implementation maturity; △ denotes partial or conditional evidence at that boundary.

System	File/blk enc.	Replay chk.	UMA policy	Boundary
Plaintext	✗	✗	✗	Lowerfs baseline only
gocryptfs	✓	✗	✗	User-space encryption baseline
fscrypt	✓	✗	✗	In-kernel file encryption
dm-crypt	✓	✗	✗	Block-device encryption
fs-verity/dm-integrity	△	✗	✗	Read-only file or block integrity
TPM/TEE-backed designs	△	✓	✗	Hardware trust boundary
GPU-storage systems	△	✗	△	Accel. path focus
AEGIS-Q	✓	△	✓	FUSE runtime with scoped evidence

Post-quantum key establishment is relevant to long-lived edge data, but it does not replace symmetric storage encryption. ML-KEM establishes shared secrets, whereas AES-GCM remains the appropriate mechanism for bulk data records [11]. SHA-256 can bind committed-prefix anchor material to persisted metadata [10]. The systems challenge is therefore to compose ordinary storage invariants—a key

hierarchy that survives remount, nonce/generation state that remains safe across crashes, ciphertext durability before metadata publication, and fail-closed recovery—with accelerator placement and shared-resource control. Table 1 positions that gap against plaintext, gocryptfs, fscrypt, dm-crypt, fs-verity/dm-integrity, TPM/TEE-backed designs, and GPU-storage systems [5, 6, 14, 19]; the missing cell is storage-visible policy at the edge encryption boundary.

Figure 1 is the paper's edge-runtime result. Unthrottled secure-storage pressure reports 9.62 ms SQLite p99 and 6.98 MB/s background writes; AEGIS-Q reports 8.15 ms p99 and 3.02 MB/s by throttling only elastic writes. The point is visible storage-layer policy with accelerator-independent format correctness, not peak filesystem throughput. The low frozen-contract throughput row is a known cost boundary, not a hidden general-purpose claim.

AEGIS-Q explores this gap as a UMA-aware secure file-encryption runtime. It adds executor-local managed-memory use and an external-anchor path that can fail closed on replayed state. The claim ties placement to format invariants, QoS policy, and recovery behavior rather than to a GPU microbenchmark or a general-purpose encrypted-filesystem replacement.

This design framing follows the lesson from our prior edge-runtime work on resource-constrained systems: a convincing edge claim must start from an explicit resource boundary and then show how a small set of mechanisms compose under that boundary, rather than reporting a single accelerator primitive in isolation [8]. In that earlier setting, the key observation was that asynchronous overlap and tiered residency only matter when they are tied to the bottleneck actually seen on low-power hardware. We apply the same discipline here for secure storage: the bottleneck is not abstract "GPU utilization" but the coupled foreground-tail, background-progress, and recovery-exposure boundary of a mounted encrypted runtime.

The central tension is that the accelerator is both a compute resource and an interference source. GPU-everything file encryption loses on small authenticated writes; KEM-per-block "full PQC" confuses the key plane with the data plane; static placement ignores UMA contention; kernel QoS controls lack file-encryption semantics. AEGIS-Q therefore follows one design insight: accelerator placement must be subordinate to storage correctness and foreground slack. AES-GCM records bind file identifier, block, generation, and length; metadata follows data/journal/checkpoint order; optional GPU work produces format-compatible results or falls back to CPU. The accelerator is an execution choice, not a security boundary.

This paper makes four contributions.

(1) **C1: Placement-safe storage format.** AEGIS-Q implements authenticated envelopes, generation-bound AEAD, durable mapping publication, and CPU/GPU rules that do not alter the persistent record format.

(2) **C2: CPU data lane, GPU/PQC maintenance lane.** Measurements explain why AES-GCM writes stay on the CPU, while independent ML-KEM envelope-refresh batches can enter the GPU lane only with slack and telemetry permission.

(3) **C3: Recovery and replay boundary.** AEGIS-Q separates replayable file witnesses from externally anchored replay-after-advance checks and labels clean-remount, tamper, stale-replay, selected app-crash, and non-claimed power-loss outcomes.

(4) **C4: Storage-visible QoS control.** AEGIS-Q includes deterministic admission, platform telemetry to mounted-FUSE throttling, and the bounded mounted SQLite result in Figure 1.

The claim boundary excludes `O_DIRECT` NVMe-to-UVM DMA, `io_uring`/eBPF completion bypass, GPU constant-time execution, persistent PCR-bound freshness, physical power-loss certification, and portability outside Jetson-style accelerated UMA edge storage.

The rest of the paper follows the structure of a systems design argument. Section 2 states the storage and UMA assumptions and the motivating CPU/GPU placement comparison, Section 3 presents the format and placement design, Section 4 records implementation choices that determine the boundary, Section 5 states the security and recovery model, Section 6 evaluates the scoped claims, and Sections 7–9 discuss implications, related work, and limits.

2 Background and Motivation

2.1 Secure storage primitives

Secure-storage systems choose a protection boundary as well as a cipher. Kernel filesystem encryption, user-space encrypted filesystems, block encryption, and TEEs occupy different boundaries and trust assumptions [5, 6, 14, 19]. Those systems make encryption deployable at mature kernel, user, block, or hardware boundaries; AEGIS-Q asks a different runtime question at the mounted edge boundary. It protects files represented by its own backing format and depends on the kernel, FUSE daemon, OpenSSL/liboqs, and the mount credential. It is therefore evaluated as a placement/QoS/recovery runtime artifact, not as a replacement for fscrypt or dm-crypt.

The cryptographic roles are deliberately distinct. AES-GCM encrypts and authenticates each data block; SHA-256 constructs the committed-prefix root used by the anchor helper; and ML-KEM-768 is a key-establishment primitive rather than a bulk-data primitive [10, 11]. We use ML-KEM only for the optional batched key-plane experiment. The production FUSE data path derives a random file data-encryption key (DEK), wraps it under a mount-derived key, and uses that DEK for AES-GCM. This avoids the atypical and expensive design of performing a KEM operation for every block.

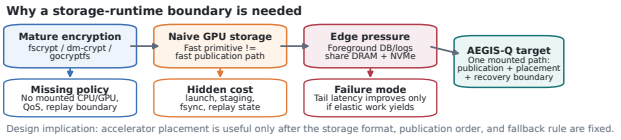

Figure 2: Motivating boundary for AEGIS-Q. The open systems problem is not encryption alone; it is the combination of authenticated publication, accelerator placement, foreground QoS, and replay-aware recovery in one mounted path.

2.2 Why a mounted runtime?

AEGIS-Q distinguishes itself by making placement and recovery policy visible at the same boundary as file encryption. It is not merely gocryptfs/fscrypt plus CUDA/TPM scripts: the research question is whether one mounted runtime can keep authenticated-block publication, storage-visible QoS/admission, optional batched GPU key-plane work, and external replay-after-advance checks under one evidence contract. The capability matrix on the first page and Figure 2 are not performance rankings. They explain why AEGIS-Q is evaluated as a placement/QoS runtime rather than as a replacement for mature deployed filesystems: existing encryption layers protect bytes, GPU-storage systems optimize data movement, and kernel controls shape I/O priority, but none of those boundaries simultaneously sees file-encryption state, accelerator slack, UMA pressure, and replay-aware recovery exposure.

2.3 Unified memory is not a DMA contract

On a unified-memory SoC, CPU and GPU share DRAM capacity and bandwidth; Jetson-class systems are a representative deployment setting [13]. This makes placement a QoS problem: a GPU job can improve an elastic batch while delaying the foreground path that shares the same memory system. CUDA managed memory makes an allocation accessible to CPU and GPU code, and prefetch is a performance hint; neither API is a permission bit nor a storage-DMA registration protocol. The present implementation allocates managed buffers only inside the CUDA executor, prefetches them around a GPU operation, synchronizes the stream before CPU reuse, and accesses backing files with ordinary `pread`/`pwrite`. It does *not* call `cudaHostRegister`, issue `O_DIRECT` into managed memory, or rely on `cudaStreamAttachMemAsync` as mutual exclusion.

This scope separates API correctness from an optimization hypothesis. Same-buffer CUDA checksums show executor visibility after a storage read; they do not prove safe registration of every file-cache page, NVMe DMA into managed memory, or deployed page-fault/coherence counter support. A future direct-I/O path would need a driver/version matrix, pinning and IOMMU evidence, memory-pressure fault injection, and a protocol that excludes DMA, CPU, and GPU concurrent access.

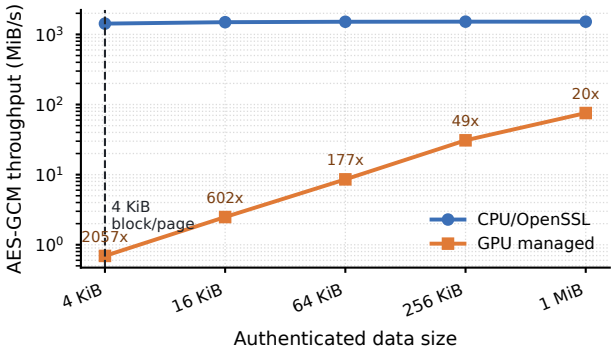

Figure 3: AES-GCM data-plane negative control from one 4 KiB authenticated block to 1 MiB. The 4 KiB tick matches AEGIS-Q's logical block and the common OS page granularity; GPU managed-buffer offload remains slower across this mounted-write range.

2.4 Motivating placement comparison

GPU acceleration is useful only when it amortizes launch, placement, and synchronization costs. Prior out-of-core systems such as FlashNeuron and Fastensor focus on moving large GPU workloads through storage hierarchies [1, 21]; their complementary staging perspectives are relevant to the batch boundary considered here [1, 21]. GPU-oriented work such as fscrypt-GPU and FlashNeuron motivates separating batch placement from the storage format [1, 22]. They do not validate this FUSE format or its durability protocol. Likewise, storage-oriented systems such as SnuQS motivate explicit staging but do not make an NVMe/UVM correctness claim for AEGIS-Q [16]. The evaluation therefore asks when a secure-storage runtime should use the CPU fast path, GPU elastic lane, or conservative fallback on the tested platform.

Figure 3 motivates the CPU side of dynamic placement. AEGIS-Q publishes data in 4 KiB logical blocks, matching the granularity at which many file and page-cache paths expose small writes. On this path, GPU AES-GCM is not merely slower at 4 KiB; it remains 20.1–2057× slower through 1 MiB because launch, managed-buffer placement, and synchronization dominate. CPU-only execution, however, leaves batched PQC throughput unused: cuPQC ML-KEM reaches 1.50 M operations/s versus 64.8 K operations/s on CPU, and the mounted refresh workflow improves from 24.575 to 20.729 ms only when slack admits the GPU. A static threshold is therefore insufficient because the crossover moves with batch shape, slack, telemetry freshness, and foreground tail latency. AEGIS-Q keeps bulk data encryption on the CPU path and admits only large, independent, slack-tolerant work into the elastic GPU lane.

2.5 Design requirements

The resulting system requirements are:

(1) **One storage format.** CPU and GPU execution must produce the same authenticated record format, so fallback does not change recovery semantics.

(2) **Publication before exposure.** A write is not visible merely because ciphertext bytes exist; a validated journal mapping and checkpoint determine exposure.
(3) **No optimistic accelerator semantics.** CUDA managed memory and prefetch are local executor mechanisms until storage-DMA behavior is separately proven.
(4) **Explicit replay boundary.** A file-backed witness is an integrity check, not rollback protection; external anchors must be evaluated separately.
(5) **Admission before acceleration.** A job with no explicit slack or stale telemetry must take the CPU path even if the GPU is present.

These requirements shape the rest of the design. They also explain the paper's evaluation style: negative controls and scoped failures are part of the evidence because they prevent the runtime from inheriting guarantees from mechanisms it does not actually implement.

3 AEGIS-Q Design

3.1 Overall procedure and component contracts

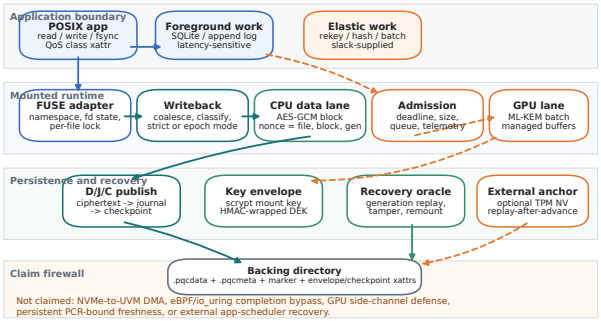

Figure 4: AEGIS-Q runtime architecture after separating the paper's three planes. Solid arrows are implemented mounted paths for foreground reads/writes, publication, recovery, and remount validation; dashed orange arrows are elastic maintenance paths that must pass admission and can fall back to CPU execution.

Figure 4 is the design map for AEGIS-Q. The section starts from this overall procedure because the paper's claim depends on which component owns each boundary, not on a collection of independent optimizations. The runtime is organized around three planes: *foreground publication*, where read/write/`fsync` becomes authenticated 4 KiB AES-GCM records and is exposed only after D/J/C publication; *elastic maintenance*, where rekey/hash batches may use the GPU only after slack-gated admission; and *recovery exposure*, where remount validates envelope, checkpoint, generation, tamper, and optional anchor state before exposing plaintext.

Reading Figure 4 from top to bottom gives the normal mounted procedure. A write enters through the FUSE adapter, is serialized against per-file state, becomes one or more CPU AES-GCM records, and remains hidden until the D/J/C publisher exposes the mapping. A read performs the inverse path after remount-time validation has accepted the envelope and checkpoint. A maintenance request, such as open-file envelope refresh, enters a separate queue: admission first checks independence, batch size, slack, telemetry freshness, and GPU queue pressure, then either runs the batch in the CUDA/cuPQC lane or returns it to the CPU. Remount starts from the bottom of the figure: the recovery oracle authenticates checkpoint state, replays only reachable journal mappings, and optionally compares the committed root with an external replay anchor before any plaintext is exposed.

Table 2 names the Figure 4 boxes that matter to the research claim. Each row states the box's owned boundary, why the box is necessary for the edge-runtime thesis, and which evaluation row closes it. This is the organizing rule for the rest of the section: subsection names are component or algorithm names from the figure, not incidental implementation topics. The table is also a filter. A mechanism that cannot be named from Figure 4 and cannot point to a closing evaluation row is implementation detail, limitation, or future work rather than a design subsection.

These names are the design units used by the implementation and evaluation; mechanisms that do not fit one of these contracts are treated as out of scope rather than hidden in the architecture. The FUSE adapter is not expanded as a separate research mechanism because it routes operations into the four substantive components rather than changing placement, publication, QoS, or recovery semantics. The section therefore follows the standard systems-paper pattern used by the prior work we model: motivation establishes why static placement fails, design starts from an overall architecture/procedure figure, the figure's components receive short contracts, and only the important boxes become mechanism subsections.

This decomposition is deliberately asymmetric. Foreground components are synchronous and format-defining; maintenance components are elastic and format-preserving; recovery components are exposure gates. Placement is therefore subordinate to storage correctness. It may change where elastic maintenance work executes, but it cannot change the durable format, nonce rule, publication order, or replay oracle. Managed memory appears only inside the CUDA executor, so placement can change maintenance locality but not the storage format or recovery contract. The following subsections therefore expand the named boxes in Figure 4: Section 3.2 defines the foreground CPU data plane and D/J/C publisher, Section 3.3 defines the elastic admission controller and GPU lane, Section 3.4 defines the storage-visible QoS controller, and Section 3.5 defines the recovery oracle and external anchor.

Table 2: Architecture-indexed component contracts used to read Figure 4.

Figure component	Owned boundary	Why it matters	Evaluation closure
FUSE adapter	Namespace callbacks, fd context, and per-file locking; routes operations into the mechanism components.	Keeps the paper at the mounted edge-encryption boundary rather than a disconnected accelerator library.	POSIX scope audit and mounted workload rows.
Foreground CPU data plane and D/J/C publisher	AES-GCM record creation, generation/AAD binding, data-before-journal-before-checkpoint exposure.	Makes CPU-first placement compatible with durable authenticated publication.	AES-GCM negative control, strict/epoch publication rows, generation and remount matrices.
Elastic admission controller and GPU lane	Slack-gated ML-KEM/hash maintenance queue with CPU fallback and executor-local managed memory.	Preserves GPU use for independent batched PQC work without moving foreground writes onto the accelerator.	cuPQC primitive row, mounted rekey workflow, break-even and fallback sensitivity.
Storage-visible QoS controller	File-class and telemetry driven throttling for elastic writes inside the mounted daemon.	Shows that placement is a storage-visible control problem under UMA pressure, not an external app scheduler.	SQLite p99/background Pareto row and admission sensitivity bundle.
Recovery oracle and external anchor	Remount exposure, tamper rejection, generation replay checks, and optional replay-after-advance anchor.	Prevents placement and QoS claims from weakening the persistent security contract.	Tamper/remount, daemon cutpoint, lower-block, file-anchor negative-control, and TPM replay-after-advance rows.

3.2 Foreground CPU data plane and D/J/C publisher

The CPU data plane expands the teal foreground path in Figure 4. Its design goal is not GPU acceleration; it is to turn ordinary mounted writes into authenticated records whose exposure point is explicit. New files receive random 256-bit DEKs and file identifiers. The key envelope authenticates the DEK before open accepts it, and the data path then encrypts only with the installed DEK.

Writes are coalesced only when they are contiguous. For each affected 4 KiB logical block, AEGIS-Q reads the current authenticated block if necessary, applies the modification on the CPU AES-GCM path, assigns a strictly increasing generation, and derives the GCM nonce from $(\mathit{fileID}, \mathit{block}, \mathit{generation})$. The file identifier, generation, logical block number, and plaintext length are AES-GCM associated data, preventing valid records from moving to different logical positions. The commit order is:

(1) write ciphertext records to `.pqcdata` and call `fdatasync`;
(2) append generation-to-offset mappings and tags to `.pqcmeta`, then call `fdatasync`;
(3) update the logical-size xattr; authenticate and write the checkpoint.

Figure 5 makes the publication protocol explicit. It is intentionally not a proof diagram: the upper path names the order that makes data reachable, and the lower path names the failure interpretations that the evaluation is allowed to claim.

The D/J/C publisher is the publication component. Its journal is the publication point: a ciphertext tail with no durable mapping is unreachable after recovery. Conversely, a mapping is written only after its ciphertext range has crossed the data durability barrier. `fsync` flushes the coalescing buffer, waits for local pending work, and calls `fdatasync` on both sidecars before returning. The lower-filesystem contract is D/J sidecar order: `fdatasync` on `.pqcdata`, then `fdatasync` on `.pqcmeta`, followed by checkpoint/xattr visibility through the same lower filesystem. Reordering after acknowledged sync or lost xattrs is outside the selected crash model; directory-entry durability is claimed only for scoped directory-`fsync` workloads.

Strict and epoch publication share one accepted-state invariant. Strict publishes data then journal/checkpoint per sync and replays the newest authenticated mapping. Epoch publishes only a complete grouped `syncfs` prefix and discards incomplete tails; it amortizes aligned writers but can lose to wait time, sparse groups, or replay work. Thus epoch changes the barrier schedule, not the nonce, AAD, or replay semantics.

The generation transition is per-file. A publish-ticket serializes reservation turns; before encryption, near wraparound fails with `EFBIG` if the affected block count would pass `UINT64_MAX`. Otherwise AEGIS-Q reserves $[\mathit{next}+1, \mathit{next}+n]$ in checkpoint/high-water state, may skip interrupted reservations, advances committed generation only after publication, recovers only mappings reachable through the validated boundary, and rejects duplicate epoch records for the same file, block, and generation. Misuse-resistant SIV-style AEAD would trade this state obligation for extra passes; AEGIS-Q instead tests older-generation replay while keeping the selected crash model explicit.

The envelope path is deliberately narrower than a full key-management system. New roots use OpenSSL scrypt in versioned `.pqc_kdf` metadata (random 32-byte per-filesystem salt; $N = 32768$, $r = 8$, $p = 1$, 64 MiB max memory); roots lacking it use the legacy PBKDF2-HMAC-SHA256 compatibility path. The password-derived mount key is never hardware-released, and open-file rekey is an HMAC envelope refresh rather than a persistent credential-rotation protocol.

Unsupported POSIX modes are part of the design boundary, not the research axis. The scope audit covers the ordinary read/write/`fsync`, scoped rename, directory `fsync`, no-open hard-link, symlink, and bounded writer subsets needed by the mounted workloads; it rejects or scopes out encrypted-file `mmap`/`msync`, open-subtree retargeting, arbitrary byte-range transactions, directory-entry crash durability, and full crash-time POSIX visibility.

3.3 Elastic admission controller and GPU lane

The elastic placement controller expands the orange path in Figure 4. AEGIS-Q keeps CPU/GPU placement dynamic,

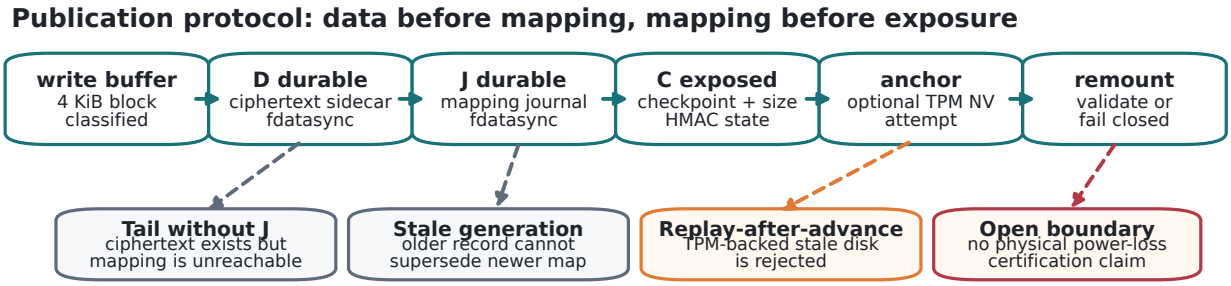

Figure 5: D/J/C/xattr publication protocol. D is the durable ciphertext sidecar state, J is the journal mapping state, and C is the authenticated checkpoint/logical-size exposure state. The bottom row is the claim boundary: tails without J are unreachable, stale generations cannot supersede newer mappings, TPM replay-after-advance fails closed, and physical power-loss certification remains outside scope.

but only for work whose semantics permit delay and fallback. Foreground AES-GCM records are immediately classified as CPU work because Figure 3 shows that 4 KiB–1 MiB managed-buffer GPU AES-GCM loses to CPU on the mounted-write range. GPU use is therefore not removed; it is constrained to the GPU lane: independent, batched, slack-tolerant maintenance work where cuPQC can amortize launch and staging.

The hardware boundary is intentionally narrow: CUDA managed allocation, prefetch, and stream synchronization are executor-local; backing ciphertext and journals still use ordinary `pread`/`pwrite` sidecars. Prototype host-pinning checks are not a storage-DMA path.

The placement algorithm is a fast predicate rather than a general scheduler. Each job first becomes foreground-sensitive data-plane work, elastic key-plane work, or unsupported optimistic offload. For a job j, AEGIS-Q chooses the GPU only when

$$\begin{aligned}\mathrm{GPU}(j) \Leftarrow\ & \mathrm{maint}(j) \wedge \mathrm{indep}(j) \wedge \mathrm{batch}(j) \geq B_{\min} \\ & \wedge\, \mathrm{slack}(j) > C_{\mathrm{launch}} + C_{\mathrm{stage}} + \Delta \\ & \wedge\, \mathrm{fresh}(\mathrm{telemetry}) \wedge Q_{\mathrm{gpu}} < Q_{\max}.\end{aligned}$$

This predicate is Algorithm 1 in the implementation: foreground data-plane jobs fail the first predicate and return CPU before taking the scheduler lock. Maintenance jobs fail closed: missing telemetry is zero budget, stale slack returns CPU, deadline and queue-pressure gates must pass, and failed predicates record a deferral reason. The error model is asymmetric: a false negative loses background progress or ML-KEM speedup; a false positive remains elastic maintenance and still produces the same persistent format after stream synchronization. No telemetry decision can move foreground AES-GCM to the GPU, skip D/J/C publication, or expose data before the recovery oracle accepts it.

The evaluated key-plane workflow applies this rule by batching open-file descriptors for ML-KEM envelope refresh, encapsulating on the selected executor, installing 32-byte refreshed DEK material, and rewriting the HMAC-authenticated envelope. It is not a persistent KEM hierarchy or hardware-backed credential lifecycle.

Algorithm 1 Elastic placement admission

Require: job j, producer slack, fresh telemetry snapshot
1: **if** j is foreground AES-GCM or publication work **then**
2: **return** CPU
3: **if** j is not independent maintenance work **then**
4: **return** CPU
5: **if** batch $< B_{\min}$ or slack $\leq C_{\mathrm{launch}} + C_{\mathrm{stage}} + \Delta$ **then**
6: **return** CPU
7: **if** telemetry is stale or GPU queue pressure exceeds $Q_{\max}$ **then**
8: **return** CPU
9: **return** GPU lane with CPU fallback on execution failure

3.4 Storage-visible QoS controller

The QoS controller is the storage-visible counterpart to admission. It handles the case where elastic work is still legal but should slow down because foreground storage is under pressure. A job enters the elastic lane only when its size and batch shape make GPU execution plausible, the producer supplies nonzero relative slack, and telemetry is fresh enough to avoid adding pressure to the foreground path. Missing, zero, or stale slack falls back to CPU with the deferral reason.

The mounted interface exposes `user.pqc_qos_class`: latency files are traced but never daemon-slept, elastic files are throttle-eligible, and SQLite sidecars inherit the base-file class. This is mounted-FUSE throttling, not an external application scheduler. Throttling can delay an elastic flush, but it cannot skip D/J/C publication, change generations, or make a GPU record visible earlier. Thus Figure 1

is a storage-visible placement result: AEGIS-Q spends background progress to reduce foreground storage-tail pressure while keeping authenticated publication unchanged.

3.5 Recovery oracle and external anchor

The recovery oracle expands the bottom half of Figure 4. On remount, AEGIS-Q authenticates the key envelope and checkpoint, replays only journal mappings whose ciphertext records are reachable under D/J/C order, checks file/block-/generation/length AAD on read, and exposes the logical size only after checkpoint validation. A tail ciphertext record without a durable journal entry is ignored; a stale or tampered authenticated record fails before data is returned.

The external-anchor path is optional and narrower than full rollback protection. The file anchor is an authenticated on-disk record. Because an attacker able to restore the backing directory can restore that record as well, it is explicitly *not* rollback protection. The crash/replay harness treats this behavior as a negative control. The optional hardware backend refuses to mount when its TPM NV index was not provisioned by an administrator; it never creates persistent TPM state implicitly. In that backend, replay after the anchor advances fails closed. These are replay-after-advance checks, not a persistent freshness lifecycle. A stronger rollback-resistance claim requires a separately protected TPM policy, defined NV authorization, a recovery protocol, and power-loss results for that exact configuration.

Table 3 summarizes the invariant boundary after the mechanisms have been introduced. Its purpose is to bind each design claim to an implementation locus and exercised path; it is not a separate feature list.

4 Implementation

AEGIS-Q is implemented in C/CUDA on FUSE3. The source tree splits the FUSE adapter, fd-context lifecycle, writeback/publication, epoch log, recovery, anchor, keyring/KDF, QoS, POSIX policy, and CUDA executors across `code/`; `pqc_fuse.c` is mount-facing. The build enables `-Wall -Wextra -Werror` for C/C++ and builds CUDA targets only when cuPQC is present.

Table 4 summarizes choices that are easy to misread. Publication is a strict data/journal/checkpoint path plus opt-in epoch-redo-log grouping, not an accelerator path. GPU AES-GCM is format-compatible and falls back to CPU, but evaluated data writes are CPU-first. The GPU ML-KEM executor is not an in-place filesystem data path; integrity parity tests do not imply a persisted per-file Merkle tree or side-channel protection.

The key-envelope path is central to remount correctness. The current path generates a random DEK, wraps it under the mount key, HMAC-authenticates the envelope, and rejects failed authentication before ciphertext is read. ML-KEM is optional batched maintenance: ordinary mounts with no rekey trigger do not allocate an ML-KEM object, generate a KEM keypair, or start the background rekey worker. They also do not preallocate the CUDA AES executor, initialize elastic admission/scheduler/QoS monitor policy, or update data-plane scheduler accounting unless rekey, QoS, telemetry, or admission tracing is configured; the external freshness anchor remains disabled unless configured; and the diagnostic GPU AES path allocates executor-owned managed buffers only if an explicit GPU batch is requested. When forced or interval rekey is configured, the worker can refresh open-file DEK material and persist new HMAC envelopes, but the format does not pretend that a KEM benchmark is a persistent per-file key hierarchy. The worker's defaults are also aligned with the elastic-lane claim: it can collect up to 128 envelopes with a 1 ms default window, skips admission setup until the candidate batch reaches the configured GPU byte gate, and suppresses successful per-batch logs unless diagnostic verbosity is requested. Thus small maintenance batches remain CPU and low-overhead, while sufficiently large batches can reach the same admission rule evaluated in Section 6.2.

The older `pqc_file_key.*` epoch/grace helper remains source-only and is not linked into `pqc_fuse`; evaluated epochs are open-file refresh bookkeeping, not stale-handle, rollback, or TPM/PCR-bound rollback proof.

The implementation also fails closed in two places. A scheduler job with no producer-supplied slack is routed to CPU. A requested TPM backend with no pre-provisioned NV index causes initialization to fail rather than creating a persistent index with undocumented ownership. These are conservative operational choices; neither substitutes for a complete external foreground-workload telemetry integration or TPM policy design.

4.1 Persistent-object layout

Each protected file is represented by ordinary lower-filesystem state rather than an in-kernel extent map. The data sidecar stores encrypted records; the metadata sidecar stores logical-block to record mappings and authentication material; xattrs store compact metadata such as logical size and the authenticated envelope. This layout is intentionally inspectable for fault-injection experiments: a harness can copy or corrupt a backing directory, restore an older sidecar, or mutate an xattr without requiring a custom block device.

The layout also creates concrete assumptions. AEGIS-Q relies on the lower filesystem to honor completed `fdatasync` order for data then metadata sidecars and to preserve checkpoint xattrs according to documented semantics. The remount oracle accepts a mapping only when referenced ciphertext, journal record, and authenticated checkpoint/xattr state are visible together. Reordering after acknowledged sync or lost xattrs is outside the selected crash model. This separates the invariant—data is synced before mapping publication—from certifying every lower-filesystem, directory, rename, drive-cache, or physical power-loss interleaving.

Table 3: Design invariants, implementation loci, exercised paths, and scope boundaries.

Invariant	Implementation locus and exercised path	Claim boundary
Envelope before DEK use	`metadata_load()` verifies the HMAC envelope before `pqc_open()`; clean remount and metadata tamper return `EKEYREJECTED`.	Remountable authenticated envelope; no hardware-bound credential release.
AEAD nonce/generation	`derive_block_nonce()`, `build_block_aad()`, and `do_flush_wbuf_locked()` bind file identifier, block, generation, and length; generation fault and clean-remount rows exercise replay.	Generation-bound record format; no physical power loss or broad workload certification.
Data-before-mapping publication	`do_flush_wbuf_locked()` syncs `.pqcdata` before `.pqcmeta`; journal, daemon cutpoint, lower-block, and SQLite rows exercise the selected D/J/C boundary.	D/J evidence only; no kernel-crash or drive-cache certification.
Checkpoint and anchor	`checkpoint_store()/load()` gate logical-size exposure and propagate hardware-anchor failures; TPM replay-after-advance fails closed and file-anchor replay is a negative control.	Authenticated checkpoint and replay-after-advance only; no persistent PCR-bound lifecycle.
Unsupported modes	FUSE `direct_io`, scoped rename, directory `fsync`, no-open hard links, symlinks, and hidden xattrs are audited.	Mounted workload subset only; no shared `mmap`, open-subtree retargeting, full link lifecycle, or full directory-entry crash durability.
Controller fallback	`pqc_block_job_choose_target()` and `pqc_admit()` return CPU on missing slack, stale telemetry, size, deadline, or queue pressure; scheduler/admission traces exercise this path.	Conservative fallback; no non-storage foreground scheduler claim.

Table 4: Implementation boundaries that are easy to misread without an explicit summary.

Mechanism	Current implementation	Boundary
CUDA storage path	Managed allocation with explicit prefetch and stream synchronization	Not an NVMe-to-UVM storage-DMA path
CUDA failure policy	CUDA executor falls back to CPU on failure	Does not change the on-disk record format
Anchor backend	File witness plus optional hardware NV index backend	Hardware path requires administrator pre-provisioning
Scheduler fallback	No-slack jobs route to CPU	No optimistic GPU admission without producer-supplied slack

4.2 Concurrency and unsupported modes

The evaluated path uses ordinary FUSE read, write, flush, fsync, open, and release operations. The implementation keeps the CPU data path separate from the elastic lane in three concrete ways.

First, foreground AES-GCM flushes bypass scheduler target construction. They do not construct write-admission contexts, read GPU telemetry, or execute a GPU admit/release branch. Runtime byte accounting follows the actual flush extent, so CPU-published AES-GCM records are not reported as GPU work and partial overwrites do not inflate GPU queue pressure by file size.

Second, disabled diagnostics are not on the common path. FUSE latency tracing, durability timing, write-throttle thresholds, telemetry polling, rekey write-trigger policy, trace sinks, parallel-commit trace snapshots, fault cutpoints, publication knobs, grouping knobs, and external-anchor backend selection are cached, initialized at mount time, or guarded by fast gates. Default mounts therefore avoid repeated environment parsing, trace open/close, per-operation timestamp collection, and disabled diagnostic atomics. The GPU-load polling thread is also lazy: it does not start unless QoS throttling, a telemetry file, an explicit GPU-load path, or an explicit monitor request is configured.

Third, repeated local work was removed from the mounted read/write path. The common path avoids same-daemon metadata reloads, sidecar EOF probes, empty-flush/release drain checks, read/write scratch allocation churn, per-open success logging, post-open policy-context relocking, redundant flush-descriptor clearing, redundant full-batch reinitialization, redundant read-side zeroing, and whole-buffer sidecar-path clearing. FUSE read/write scratch is ordinary CPU memory; CUDA managed allocation and prefetch are confined to executor-owned buffers and explicit diagnostics.

Strict publication still drains coalesced writes before sync returns. It updates sidecar dirty epochs and D/J append cutpoints at batch boundaries while recording per-block ciphertext offsets before journal mapping preparation. When no external anchor backend is configured, checkpoint xattr verification skips committed-prefix map construction, anchor-worker staging, and fsync-time anchor flush; same-size strict overwrites reuse the already-reserved final checkpoint instead of rewriting identical xattr bytes after journal publication. Epoch mode can group selected publication barriers while replaying only committed prefixes. These choices are performance engineering inside the scoped runtime contract; they support the retained remount and SQLite selected-boundary experiments, not a general high-throughput filesystem claim.

The implementation is not presented as a full POSIX conformance layer. In particular, the paper does not claim validated shared `mmap` semantics, all possible concurrent-writer races, cross-process byte-range lock compatibility, or arbitrary rename/directory-fsync crash behavior. Those modes are common sources of filesystem bugs, and the review correctly identified them as missing evidence. Treating them as unsupported or unvalidated is preferable to silently inheriting a POSIX claim from FUSE.

4.3 Telemetry plumbing

The admission logic is implemented as ordinary C state rather than a separate daemon that invents scheduling

Table 5: Threat classes and mechanism boundaries.

Threat	Mechanism	Boundary
Backing-store read	AES-GCM ciphertext with per-file DEK	Assumes mount credential remains secret
Backing-store tamper	AEAD tags, HMAC envelope, HMAC checkpoint	Denial of service remains possible
Whole-directory replay	Optional external TPM anchor can fail closed after advance	File-only witness is replayable; full rollback resistance is not claimed
Crash during update	Ordered publication plus selected daemon/app cutpoint tests	Physical power-loss and drive-cache certification remain open
GPU side channel	No protection mechanism claimed	Integrity parity is not constant-time evidence
Privileged local attacker	Out of scope	Kernel/driver/FUSE process are trusted

results. It records input queue depth, batch size, slack, telemetry age, and route decision. On the elastic path, cumulative route counters and telemetry samples are read through atomic snapshots, and trace emission first checks a cached trace-enabled gate before taking the trace lock. The mounted-FUSE throttle path reads a telemetry file and applies hysteresis before sleeping inside the write-flush path. The tegrastats and CUPTI harnesses differ only in how they populate that telemetry input. This shared path is the reason the evaluation can claim same-run telemetry-to-FUSE wiring; it is still a storage-runtime result rather than an external application scheduler.

5 Security and Recovery Scope

5.1 Threat model

AEGIS-Q considers an attacker who can read, copy, modify, and replay the backing directory but cannot obtain the mount credential or compromise the running kernel, FUSE daemon, CUDA driver, or cryptographic libraries. Under that scope, AES-GCM protects the confidentiality and authenticity of published block records, while the HMAC-protected key envelope and checkpoint detect unauthenticated metadata changes. The adversary can still deny service, delete records, replay a complete file-backed snapshot, or exploit vulnerabilities outside the FUSE process. We make no claim against a privileged local attacker that can read process memory, substitute the CUDA driver, alter the mount executable, or use GPU microarchitectural side channels.

The accepted encrypted state consists of a validated DEK envelope, a complete journal mapping, and ciphertext records referenced by that mapping. A tag failure causes a read to fail. A malformed or unauthenticated envelope/checkpoint causes open to fail. Table 5 states the threat classes these mechanisms address and the boundaries left outside the paper. These behaviors are necessary for a safe FUSE format, but they are not a formal proof of crash consistency. In particular, xattr atomicity and directory durability are delegated to the underlying filesystem.

5.2 Replay protection is conditional on an external anchor

The source tree contains two anchor modes. The file mode is only an integrity witness; copying the directory copies the witness. The corrected crash/replay run therefore reports four of four file-backed snapshot restores as *rollback visible*, not as successful protection. This negative result is important: an HMAC alone does not prevent rollback when its entire state is replayable.

The hardware mode stores an authenticated prefix record in an owner-authorized TPM NV index: administrators create and rotate the index outside the daemon, and AEGIS-Q reads or writes only that record. PCR binding is a transient probe rather than a persistent filesystem policy; a software update or PCR change therefore requires explicit reprovisioning/resealing. Backup or migration cannot copy only the backing directory; it needs a fresh authorized anchor bootstrap. Lost mount credentials are unrecoverable. Missing index, authorization failure, TPM I/O failure, digest mismatch, or a TPM sequence ahead of local state fail closed; a TPM anchor behind local state is treated as a lagging witness and must be refreshed rather than as disk rollback. The evaluated claim is only replay-after-advance fail-closed behavior, including the SQLite same-store campaign in Section 6.3; it does not assert that an NV index is a signer, report invented TPM latency, or claim persistent PCR binding.

5.3 Side channels and application semantics

GPU SIMT execution, cache behavior, memory coalescing, occupancy, and contention can all leak information. Simulated timing and TVLA traces are excluded from this paper. The GPU integrity test checks output parity with OpenSSL; it says nothing about constant-time execution, physical power/EM leakage, or cross-tenant GPU channels.

Likewise, the clean-remount test validates one encrypted 128 KiB round trip through the final binary. It does not certify SQLite WAL, `mmap`, locking, crash recovery across every cut point, or the complete POSIX contract. The current security evidence is local to the edge runtime: FUSE self-tests, clean remount, managed-memory smoke, and GPU integrity parity show the implemented boundary, not statistical security, cloud/privacy economics, or remote key-management behavior.

Application semantics are separated from storage-record integrity. AEGIS-Q can reject a corrupted block tag, but it cannot infer whether an application-level transaction was acknowledged unless the experiment supplies an external oracle. The daemon power-fault matrix adds final-binary `SIGKILL` at selected one-block storage boundaries, while the SQLite rows supply application commit oracles. A complete database certification would need broader workloads, multiple cache modes, physical power loss, kernel crash, and drive-cache timing.

6 Evaluation

6.1 Methodology and claim scope

Following the prior-paper style, the evaluation is organized as *boundary first, mechanism second, scope third*. RQ1–RQ2 test the main edge-runtime thesis, while RQ3–RQ5 close correctness, recovery, and cost boundaries that keep the thesis honest. It answers five research questions:

- **RQ1: CPU/GPU/PQC placement.** When should a UMA edge runtime keep AES-GCM file encryption on the CPU, and when can ML-KEM/cuPQC key-plane maintenance use the GPU?
- **RQ2: Mounted QoS and edge-storage behavior.** Does storage-visible admission recover SQLite latency under secure-storage pressure, and do scoped append-log/cache-manifest workloads survive remount?
- **RQ3: Correctness of authenticated publication.** Does the implemented format preserve nonce/generation, publication, and metadata-auth invariants under remount, fault, and tamper campaigns?
- **RQ4: Replay and recovery oracle.** Which daemon/app crash and TPM replay-after-advance outcomes are supported, and which rollback or power-loss claims remain outside scope?
- **RQ5: Cost boundaries and sensitivity.** What strict-publication cost appears against mode-aligned baselines, and which controller/key-plane/anchor mechanisms account for the result?

The design/evaluation contract is one closure per mechanism: placement by data-plane negative controls plus key-plane workflow; QoS by mounted SQLite recovery plus kernel-control rows; publication/recovery by generation, tamper, and cut-point crash campaigns; and replay by file witness versus TPM replay-after-advance. Primitive-placement numbers come from dedicated microbenchmarks, while mounted behavior comes from workflow harnesses. The retained pipeline regenerates figures/tables from structured JSON outputs rather than manual copy. Workloads remain intentionally scoped (SQLite, append-log, cache-manifest, bounded multi-writer), not a broad service benchmark suite.

The platform was an NVIDIA Jetson AGX Thor Developer Kit with 14 ARM CPU cores, a 1 TB WD PC SN5000S NVMe device, Linux 6.8.12-tegra, CUDA 13.0.48, liboqs 0.15.0, FUSE3 3.14.0, and cuPQC; it is not an Orin Nano result. We use Thor as a representative Jetson-class accelerated UMA platform for the placement/QoS question, not as evidence of cross-SoC portability. Retained O4 tegrastats logs give same-run power/thermal observations: mean VIN is 29.1 W for SQLite QoS, 27.5 W for key-plane rekey, and 28.4/27.4 W for AEGIS-Q/dm-crypt frozen rows, with max `tj`=49.0C. These are observations, not energy-efficiency claims.

Figure 6 is the spine: CPU-first data plane, slack-gated GPU key plane, and storage-visible QoS recovery with explicit throughput tradeoff. The strict frozen-contract row is a cost boundary, not the headline. Its contract fixes 4 KiB 70/30 random read/write, `psync`, per-write `fdatasync`, queue depth 1, one client, 1 GiB, fixed mounts/platform, and five repetitions. A same-profile attribution probe confirms that strict publication still pays data-sidecar, journal-sidecar, and marker/checkpoint durability boundaries. The matched dm-crypt row uses the same warm-cache contract (9.03 MiB/s median; 264/823 μs p99/p99.9), while fscrypt remains environment-blocked by missing kernel/filesystem support.

X6/O3 test strict-path practicality without changing crash semantics. X6 replaces filesystem-wide `syncfs` with marker-file `fsync` while preserving data/journal barriers; O3 allows epoch grouping only for concurrent batches that can safely share a barrier. Grouping improves four-client p99 (47.6→40.1 ms) and throughput (9.05→9.72 MiB/s), but fault-matrix rows still expose regressions under some patterns. We therefore treat it as conditional admission, not a universal fast path.

6.2 Dynamic placement: CPU data and GPU ML-KEM

The diagnostic placement harness explains the key asymmetry behind Figure 6(a) and Figure 3. For a 16 MiB AES-256-GCM input, CPU/OpenSSL reaches 1.61 GB/s (observed 1.59–1.63 GB/s), whereas the managed-buffer GPU path reaches 0.172 GB/s (0.172–0.172 GB/s). The GPU result includes managed-buffer staging and stream synchronization. The retained 4 KiB–1 MiB data-plane negative-control rows amplify the same effect, with the current CUDA path 20.1–2057× slower than CPU. This is why AEGIS-Q's data plane is CPU-first; a GPU label alone is not a performance result.

For a batch of 4,096 ML-KEM-768 key generations, liboqs on CPU reaches 64.8 K operations/s (64.3–64.9 K), while cuPQC reaches 1.50 M operations/s (1.49–1.51 M), a 23.2× ratio of medians. The comparison is a primitive microbenchmark on one platform, not a claim that a FUSE write becomes 23.2× faster. It supports the narrower policy of reserving GPU work for sufficiently large, independent key-plane batches.

The mounted ML-KEM-768 key-plane workflow opens 1,024 encrypted files and refreshes their HMAC-protected envelopes. In the five-run methodology row, CPU-only refresh takes 24.575 ms (41.7 K files/s); with explicit slack, admission selects the GPU and finishes in 20.729 ms (49.4 K files/s), a 1.186× median speedup; with zero slack and high memory-pressure telemetry, fallback remains CPU-equivalent at 24.572 ms. The same controller treats missing or stale slack as CPU fallback rather than extrapolating from old telemetry; the sensitivity bundle in Section 6.4 varies sampling, thresholds, queue depth, budget, and background intensity to expose the tuning envelope. The X11 break-even model explains the bounded workflow gain: primitive cuPQC

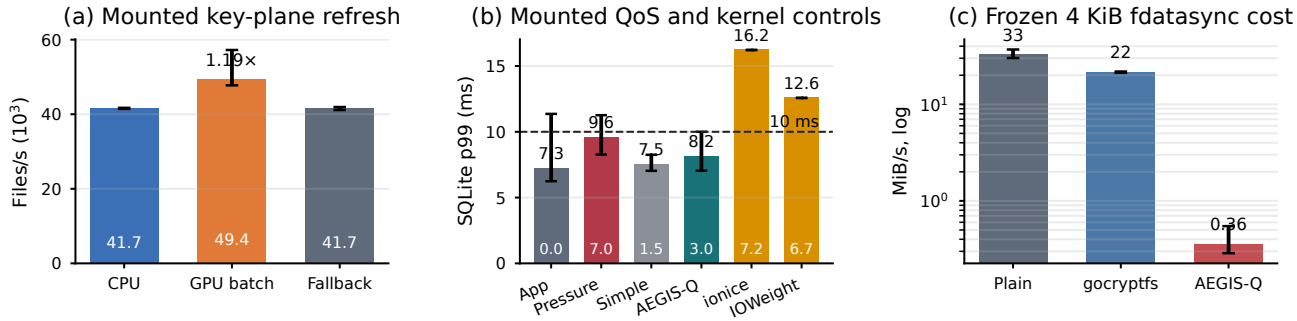

Figure 6: Evaluation summary generated from retained validation JSON. Panel (a) shows the mounted open-file key-plane refresh workflow for RQ1; Panel (b) shows mounted SQLite QoS and kernel-control rows for RQ2; Panel (c) shows the strict frozen-contract cost boundary for RQ5.

is faster by batch 16, but mounted envelope/FUSE/admission overhead shifts the positive workflow break-even to roughly 760–812 refreshed files, and an added 5 ms of UMA or staging pressure shifts it to roughly 1.09–1.10K files. The result is therefore a maintenance-lane placement boundary, not a claim that PQC acceleration makes ordinary file writes faster, provides hardware-backed credential release, creates a persistent KEM hierarchy, or recovers non-storage foreground latency.

These measurements produce the main placement rule used by the design: AEGIS-Q keeps the CPU for bulk data because AES-GCM data writes are latency-sensitive, while ML-KEM batches are elastic and may use the GPU only when producer slack and telemetry permit it. The result would be weaker if the paper reported only the faster GPU primitive; the important systems observation is the boundary between the data plane and the slack-tolerant maintenance plane.

6.3 Scoped edge-storage workloads and negative controls

The GPU integrity test passes leaf, Merkle, and empty-tree parity cases against OpenSSL. The FUSE self-test covers journal/checkpoint/anchor units and a generation-replay regression. The retained generation fault matrix covers partial update, clean remount, torn tails, stale snapshots, journal loss, older-generation append, and duplicate-generation replay across strict and epoch modes; every row has an oracle verdict without silent-corruption or unexpected-liveness outcomes. X10 adds `UINT64_MAX`/`EFBIG` near-wrap refusal, publish-ticket reservation, and same-file/disjoint 4-thread/4-process mounted writer stress, but not arbitrary application-level concurrency. Clean remount byte-compares plaintext after `fsync`, and envelope tamper returns `EKEYREJECTED`.

The POSIX-scope audit records a narrow mounted-workload interface: ordinary read/write/`fsync`, scoped rename, directory `fsync`, no-open hard links, symlinks, open-fd truncate/fallocate, user xattrs, and bounded writer races pass without mixed plaintext. Encrypted-file `mmap`/`msync`, open-subtree retargeting, arbitrary byte-range transactions, lower-filesystem xattr atomicity, and full crash-time POSIX visibility are rejected, scoped, or delegated.

Auxiliary storage/GPU diagnostics are deliberately scoped. The host-pinning and managed-storage probes show same-buffer CPU/GPU checksum agreement plus CUDA pointer diagnostics, but they do not prove UVM migration suppression or final FUSE data-path DMA.

The file-backed anchor produces the deliberately negative result: restoring an entire backing-directory snapshot is classified as `rollback_visible`, i.e., the previous committed baseline remains readable. The result remains a negative control rather than rollback resistance.

The TPM runs record anchor latency, PCR reads, NV public attributes for index `0x01500010`, and a transient PCR-policy probe. The hardware replay matrix labels stale disk/new TPM, new disk/stale TPM, missing index, changed PCRs, authorization failure, interrupted NV update, and normal replay-after-advance. This is replay-after-advance evidence, not a claim that the persistent filesystem anchor itself is PCR-sealed.

The broader crash/replay matrices are selected-boundary evidence, not complete app-level crash coverage. The daemon campaign kills the final binary at D/J/C, anchor, `fsync`, remount, and authenticated-read cut points; application rows cover SQLite and `dbm.dumb` TPM replay. X1 adds three loopback ext4/device-mapper lower-block `error` rows; after `e2fsck`/remount, pre-commit rows recover the previous committed payload and the post-`fsync` row recovers the latest committed payload. These rows interrupt block availability below the D/J/C contract; they are not a proof of device-cache flush ordering, physical power loss, kernel crash, arbitrary workloads, or multi-block raw-write behavior.

The scoped mounted-application rows are not synthetic fio rows. The secure append-log macro appends ordered 16 KiB records in strict and epoch modes, remounts, and verifies hashes/order; retained p99 is 5.93 ms strict and 4.17 ms epoch. A cache-manifest workload publishes 24

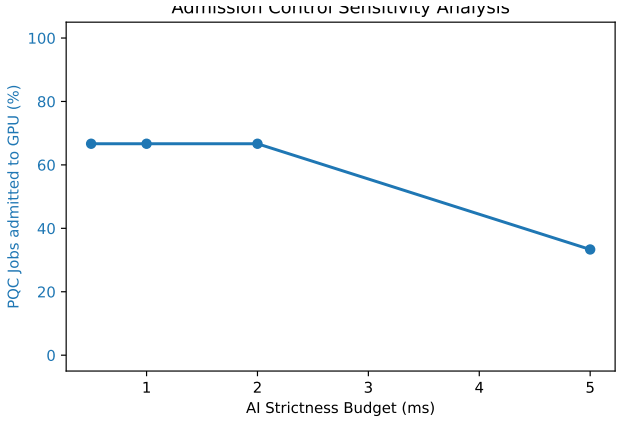

Figure 7: Admission sensitivity to foreground strictness budget (RQ2/RQ5 diagnostic). Tighter latency budgets reduce GPU-admitted maintenance work, enforcing conservative fallback under pressure.

hashed 16 KiB objects with closed-file rename plus directory `fsync`, remounts, and verifies all hashes, reporting 4.22/28.09 ms p50/p99 at 2.00 MiB/s. The multi-writer row shows the queue-depth limit: epoch grouping reduces traced syncs from 16 to 1 and replay reconstruction remains sub-ms, but p99.9 worsens to 67.90 ms versus 61.54 ms. These are scoped mounted patterns, not a broad workload suite or non-storage application QoS claim.

Recovery/replay outcomes support the mounted runtime boundary; they are not the paper's central QoS/placement result.

6.4 QoS recovery and admission sensitivity

The tegrastats/CUPTI runs show platform pressure reaching the mounted daemon: they record 8 and 5 daemon-throttled flushes, respectively. This remains wiring evidence rather than application-level p99 recovery or fast-notification bypass.

The app-level result uses SQLite transaction latency because it exercises the mounted file-encryption path, foreground tail latency, and elastic background pressure together. Table 6 reports five-run hero medians and three-run kernel QoS controls. AEGIS-Q reports 8.15 ms p99, zero median misses, and 3.02 MB/s background progress; unthrottled storage reports 9.62 ms and 6.98 MB/s. This is a Pareto point rather than a free throughput improvement: the simple controller reaches 7.54 ms p99 but only 1.50 MB/s background progress. Kernel controls preserve background throughput but not the tail here: `ionice` reports 16.22 ms p99 with six misses, and `IOWeight` reports 12.57 ms p99 with seven misses. This is bounded storage-visible control, not a uniqueness or non-storage QoS claim.

The sensitivity bundle varies budget, sampling interval, threshold, queue depth, and writer intensity (Figure 7, left). Defaults are 20 ms sampling, 0.70/0.60 enter-exit thresholds,

Table 6: SQLite p99 under mounted secure-storage pressure. Hero rows are five-run medians; kernel rows are 3 retained runs each.

Row	Control	p99	Miss	MB/s	Evidence
App only	none	7.25	0.00	0.00	5-run med.
Unthrottled	none	9.62	0.00	6.98	5-run med.
Simple ctrl.	harness sleep	7.54	0.00	1.50	5-run med.
AEGIS-Q	storage class	8.15	0.00	3.02	5-run med.
Kernel ionice	`ionice -c 3`	16.22	6.00	7.25	3-run proof
Kernel IOWeight	`IOWeight=10`	12.57	7.00	6.73	3-run proof

two-sample hold, 128 KiB minimum GPU-byte gate, 1.73 ms minimum producer slack, 100 μs deadline margin, 250 ms telemetry-age limit, and 0.80 queue threshold. A 4 KiB foreground record is never promoted just because admission exists. Baseline p99 is 6.4 ms; 80 ms sampling and 128 KiB chunks remain near 7.0/7.2 ms, high threshold disables throttling (8.8 ms), and two writers are fragile (12.3 ms). Misclassification cost remains asymmetric: conservative decisions lose elastic progress, while aggressive decisions raise SQLite tail latency. This keeps the claim on storage-visible control, not external application scheduling.

The ablation manifest then attributes cost to the authenticated FUSE format, SQLite recovery to elastic-write throttling, key-plane speedup to slack-gated GPU maintenance, and replay refusal to the external anchor.

7 Discussion and Limitations

The useful result is asymmetry, not aggregate "GPU speedup." CPU/OpenSSL fits AES-GCM data writes, while GPU/cuPQC helps slack-tolerant ML-KEM maintenance. AEGIS-Q trades peak background throughput for explicit publication, placement, QoS, and recovery boundaries; stale telemetry costs elastic progress, not the CPU/D/J/C foreground path.

The trust model remains kernel/FUSE-daemon trust plus backing-format checks. The mount password is the root credential; privileged-local attacks, GPU side channels, and hardware-backed key release are outside the current claim. TPM support is limited to replay-after-advance fail-closed behavior, not persistent PCR-bound lifecycle protection.

Deployment scope is intentionally narrow. We evaluate read/write/`fsync`, scoped rename, directory `fsync`, and bounded writer races, but not shared `mmap`/`msync`, full POSIX edge cases, broad application concurrency, or full cross-SoC portability. The crash matrix covers selected daemon/app cut points and loopback lower-block faults, not physical power-loss or kernel-crash certification.

Future hardening follows three tracks: (i) stronger rollback lifecycle (persistent PCR policy and anchor evolution), (ii) wider QoS and workload comparison (additional kernel I/O controllers and non-SQLite workloads), and (iii) broader platform validation plus side-channel-aware GPU paths. Kernel-path acceleration (e.g., `io_uring` or fscrypt-adjacent hooks) is viable only if the same authenticated publication and recovery oracle are preserved.

8 Related Work

AEGIS-Q separates storage protection, GPU/storage placement, and PQC key-plane work. fscrypt and dm-crypt are mature kernel baselines [5, 6], and gocryptfs is the closest user-space encrypted-filesystem comparison point [19]. The mode-aligned measured rows are plaintext/lowerfs, gocryptfs, dm-crypt, and AEGIS-Q; they expose publication and QoS costs. fscrypt is unavailable with proof, not an implied speedup, and gocryptfs marks a loss because AEGIS-Q targets placement/QoS/replay rather than kernel-replacement maturity.

Our prior edge systems paper (EdgeQuantum) studied a different workload class (quantum-circuit simulation) but used a similar edge methodology: identify the dominant resource boundary first, then evaluate whether asynchronous overlap and memory-hierarchy policy actually move that boundary on constrained hardware [8]. AEGIS-Q reuses that methodology pattern while changing the contract: from simulation capacity/latency to authenticated publication/recovery and storage-visible QoS. This continuity is methodological, not a claim that secure mounted writes and quantum simulation share the same mechanism or threat model.

GPU/storage systems validate different contracts. FlashNeuron/Fastensor stage large GPU workloads [1, 21]; SnuQS manages out-of-core capacity [16]; Speculative GPU encryption and fscrypt-GPU optimize throughput-oriented encryption paths [4, 22]; and GPUstore, GPUfs, GeminiFS, GPU4FS, and GPUDirect Storage expose GPU-side execution, storage, or direct movement [9, 12, 17, 18, 20]. They motivate pressure but not AEGIS-Q's FUSE records, anchors, D/J/C publication, TPM replay, storage-visible QoS, SPDK, cuFile, or a direct storage/GPU data path.

Linux storage QoS mechanisms such as blk-throttle, `io.latency`/`IO.cost`, BFQ, and CFQ operate below file-encryption semantics; AEGIS-Q's `ionice`/`IOWeight` rows are controls, not an exhaustive scheduler study. F2FS, ScaleFS, FastCommit, encrypted databases, integrity trees, and secure logs place append/replay oracles at other boundaries. ML-KEM is not per-block encryption [11]; post-quantum encrypted-file-system cost models reinforce that primitive choice is not a mounted runtime [2]. OP-TEE/TrustZone isolates attested code paths [14]; AEGIS-Q does not provide that TEE boundary or persistent rollback policy.

Additional baselines clarify scope. eCryptfs and ZFS-native encryption represent mature encrypted-filesystem boundaries, and FSCQ represents stronger formal crash-consistency assurance [3, 7, 15]. AEGIS-Q does not claim that semantic completeness; it contributes a narrower runtime boundary where placement/admission is co-managed with authenticated publication and replay-labeled recovery. Likewise, freshness/rollback and GPU-side-channel protection remain boundary-dependent hardening tasks beyond the current replay-after-advance claim.

9 Conclusion

AEGIS-Q treats secure edge file encryption as a placement, QoS, and recovery-boundary runtime under secure-storage pressure, not blanket GPU offload. Its format keeps AES-GCM writes on CPU, admits ML-KEM/cuPQC maintenance only with slack, and reports 8.15 ms SQLite p99 with 3.02 MB/s background progress on Thor. Accelerator throughput matters only after publication order, QoS, and replay boundaries are explicit.

References

[1] Jonghyun Bae, Jongsung Lee, Yunho Jin, Sam Son, Shine Kim, Hakbeom Jang, Tae Jun Ham, and Jae W. Lee. 2021. FlashNeuron: SSD-Enabled Large-Batch Training of Very Deep Neural Networks. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*. 387–401.

[2] Daniel J. Bernstein, Tung Chou, Tanja Lange, and Christine van Vredendaal. 2024. Predicting Performance for Post-Quantum Encrypted-File Systems.

[3] Haogang Chen, Daniel Ziegler, Tej Chajed, Adam Chlipala, M. Frans Kaashoek, and Nickolai Zeldovich. 2015. Using Crash Hoare Logic for Certifying the FSCQ File System. In *Proceedings of the 25th Symposium on Operating Systems Principles (SOSP 15)*.

[4] Vandeir Eduardo, Luis C. Erpen de Bona, and Wagner M. Nunan Zola. 2019. Speculative Encryption on GPU Applied to Cryptographic File Systems. In *17th USENIX Conference on File and Storage Technologies (FAST 19)*. 93–105.

[5] Clemens Fruhwirth. 2005. New methods in hard disk encryption: dm-crypt and LUKS. *Linux Journal* 2005, 138 (2005).

[6] Michael Halcrow, Theodore Ts'o, et al. 2015. Ext4 Encryption: Design and Implementation of Linux Filesystem Encryption. *Proceedings of the Linux Symposium* (2015).

[7] Michael A. Halcrow. 2007. eCryptfs: An Enterprise-class Encrypted Filesystem for Linux. In *Proceedings of the Linux Symposium*.

[8] Sunggon Kim and collaborators. 2026. edgeQsim: Quantum Circuit Simulation Framework for Resource-Constrained Edge Devices. Prior paper in the project archive (Paper/Previous paper/previous paper.pdf).

[9] Julian Maucher, Steven Diethardt, Lukas Steffen, Alexander Girol, Harald Obermaier, and Andreas Koch. 2024. GPU4FS: Full-Scale File System Acceleration on GPU. In *Architecture of Computing Systems (ARCS 24)*.

[10] National Institute of Standards and Technology. 2015. *FIPS 180-4: Secure Hash Standard (SHS)*. Technical Report. NIST.

[11] National Institute of Standards and Technology. 2024. *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard*. Technical Report. NIST.

[12] NVIDIA Corporation. 2026. *NVIDIA GPUDirect Storage: cuFile API Reference Guide*.

[13] NVIDIA Corporation. 2026. *NVIDIA Jetson Linux Developer Guide: Platform Power and Performance*.

[14] OP-TEE. 2018. OP-TEE: Open Portable Trusted Execution Environment Specification. In *GlobalPlatform Technology Conference*.

[15] OpenZFS Project. 2024. *OpenZFS Native Encryption*.

[16] Daeyoung Park, Heehoon Kim, Jinpyo Kim, Taehyun Kim, and Jaejin Lee. 2022. SnuQS: Out-of-Core Cache Staging for Virtualized Memory Storage Systems. In *Proceedings of the International Conference on Supercomputing*.

[17] Shi Qiu, Weinan Liu, Yifan Hu, Jianqin Yan, Zhirong Shen, Xin Yao, Renhai Chen, Gong Zhang, and Yiming Zhang. 2025. GeminiFS: A Companion File System for GPUs. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. 221–236.

[18] Mark Silberstein, Bryan Ford, Idit Keidar, and Emmett Witchel. 2013. GPUfs: Integrating File Systems with GPUs. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 13)*. 485–498.

[19] Jakob Spenneberg et al. 2017. gocryptfs: A Modern FUSE-based Encrypted Filesystem. In *USENIX Security Association*.

[20] Weibin Sun, Robert Ricci, and Matthew L. Curry. 2012. GPUstore: Harnessing GPU Computing for Storage Systems in the OS Kernel. In *Proceedings of the 5th Annual International Systems and Storage Conference (SYSTOR 12)*.

[21] Jia Wei, Xingjun Zhang, Longxiang Wang, and Zheng Wei. 2023. Fastensor: Optimize the Tensor I/O Path from SSD to GPU for Deep Learning Training. *ACM Trans. Archit. Code Optim.* 20, 4 (2023).

[22] Xingjun Zhang, Longxiang Wang, et al. 2023. fscrypt-GPU: GPU-Accelerated Filesystem Encryption for High-Throughput Secure Storage. In *ACM Symposium on Cloud Computing*.